

Agentic Workflows

Service-Enabled Science — Session 05: giving the agent tools, not just a chat window

Huihuo Zheng

Argonne Leadership Computing Facility

30 September 2026



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

From services to agents



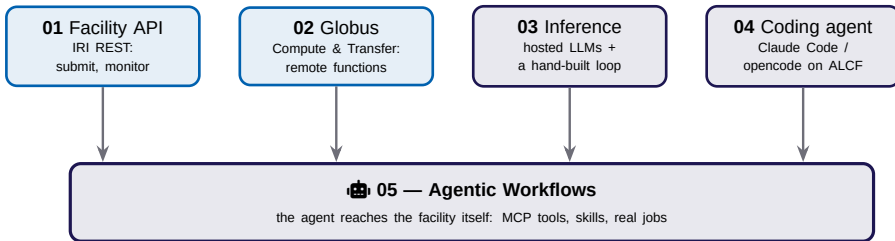
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Sessions 01–04 built the pieces

This session is where they get assembled into one agent



Same services, new caller. In 01–03 *you* made the calls; here the agent does — under your token, with you watching.

The bottleneck is mechanics, not science

Where a scientist's time actually goes before the first result

>_ Manual workflow today

1. SSH to a login node
2. Read module docs, hand-write a job script
3. Fix account, queue, walltime, paths
4. Submit, poll qstat
5. Parse errors, edit, resubmit
6. Move output files by hand

Hours of mechanics before the science starts.

Agentic workflow

“Run my training script on 2 Polaris nodes and tell me when it converges.”

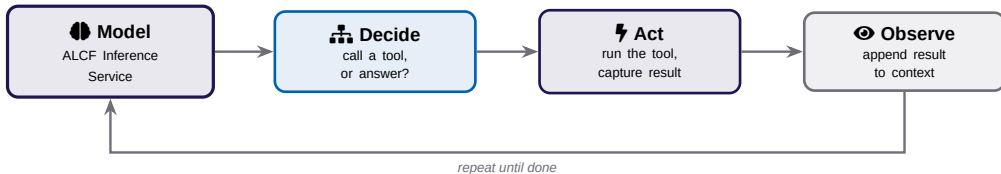
- Agent resolves account, queue, paths
- Stages the script, submits the job
- Monitors; diagnoses failures
- Reports results with evidence

The scientist stays in the loop and owns the science.

The mechanics are the bottleneck — and ALCF's service layer is what makes them programmable.

An agent is a loop

A model, a set of tools, and a stopping condition — nothing more



</> You already wrote one

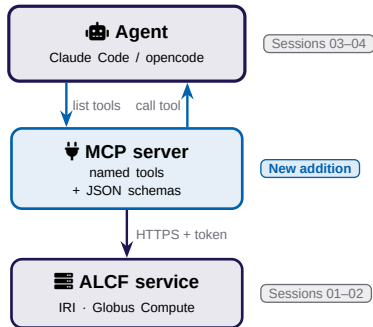
Session 03's `04_agent_loop.py` is exactly this loop in 198 lines, with one tool: `get_system_status`. Run it with `--step` to watch each turn.

🔧 What is missing is the tools

The loop is the easy part. The value is in *which* tools the model can reach — and whether they are safe to hand it.

MCP turns a service into a tool

Discovered at runtime, typed, and scoped by you



You built the top and bottom in Sessions 01–04. Today adds the middle.

The MCP server is where your judgment lives: what you do not expose, the agent cannot do.

🔍 Why not just paste the docs?

- ✓ **Discovered, not hard-coded.** The agent asks the server what it can do; add a tool, no prompt edit.
- ✓ **Typed.** Every tool has a schema, so arguments are validated before anything runs.
- 🔒 **Scoped.** The server is the boundary — it can refuse a path, a queue, or an account.

The session-05 toolkit



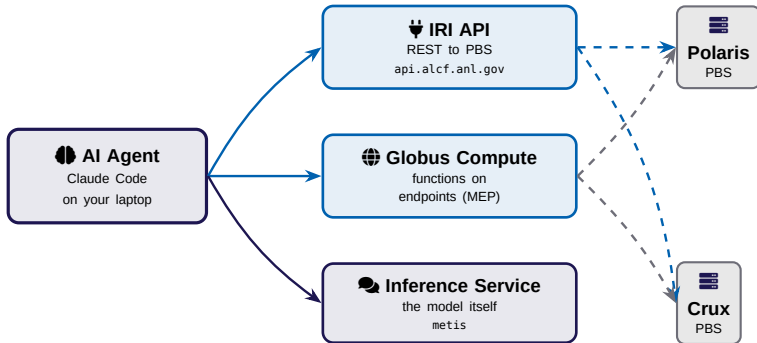
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Every service today is an agent tool

The same endpoints from Sessions 01–03, now reachable by name



One alcf-tokens login mints every token the agent needs — Globus OAuth2, short-lived, and yours.

Point your agent at the Inference Service

Three commands — and none of them are Anthropic-specific

🔑 What each step does

1. **Authenticate once.**

Mints the inference, IRI and Globus tokens together.

2. **Configure the agent.**

Writes `~/.claude/settings.json`.

3. **Launch it.**

🔄 Re-run step 2 to refresh an expired key.

>_ setup

```
alcf-tokens login \  
  --authorize-transfer home \  
  --authorize-transfer eagle  
  
uvx alcf-ai agent configure claude  
# ... or: configure opencode  
  
claude
```

`./setup.sh` installs the CLIs and the Python env — you still run step 1 yourself.

🔧 Hands-on 3 also needs **Globus Connect Personal**. It wants a browser login, so install it *before* the session.

Start with a server you did not write

ALCF already publishes its user guides as an MCP endpoint

 .mcp.json --- no token

```
{
  "mcpServers": {
    "ask-alcf": {
      "type": "http",
      "url": "https://ask.alcf.anl.gov/mcp"
    }
  }
}
```

The URL is the whole configuration — nothing to install, nothing to authenticate.

 **opencode:** Cloudflare 403s it by TLS fingerprint. Run `ask_alcf_proxy.py` instead — a server that is also a *client* of another.

🔍 One tool, and why it earns its place

- 📖 `retrieve_alcf_docs(query, top_k)` — ALCF, OLCF, NERSC and LLNL guides, plus PBS, Slurm and CUDA.
- 🔗 Returns passages **with source URLs**, so you can check what the model was actually told.
- ❓ “*Walltime on Polaris debug?*” — it reads the policy, not its memory.

Not every server is yours to build — point at one that exists, then compose.

Now write your own

It wraps the same Session-01 REST calls you made by hand

 `alcf_mcp.py` --- one tool

```
@mcp.tool()
def submit_job(system: str, commands: str,
               stdout_path: str, nodes=1,
               queue="debug",
               account="alcf_training",
               walltime_sec=600) -> dict:
    """Submit a PBS job, return its ID.

    Refuses accounts outside the
    workshop allocation, >2 nodes, or
    walltimes over 30 minutes.
    """
```

The docstring is the tool description the model reads.

☰ Six IRI tools

- 📡 `get_system_status` — Polaris / Crux up?
- ⚡ `submit_job` — PBS job via IRI
- 🔄 `get_job_state` — poll one job
- ☰ `list_jobs` — filter by state / queue
- ✕ `cancel_job` — stop a run
- 📁 `read_file` — fetch stdout / stderr

Six REST endpoints from `01_Facility_API/`, now callable by name — and four more on the next slide.

Four more tools: get your own files there

Sessions 01–04 only touched files already at the facility

 `alcf_mcp.py --- staging`

```
@mcp.tool()
def stage_to_alcf(local_path: str,
                  alcf_path: str,
                  recursive=False) -> dict:
    """Copy a local file to ALCF.

    Returns as soon as Globus accepts
    the task -- the bytes have NOT
    moved yet. Poll transfer_status
    until SUCCEEDED before submitting.
    """
```

≡ The other three

- 🔌 `local_endpoint` — your GCP collection ID, so no UUID sits in the repo
- 📁 `globus_ls` — local or alcf; look before you invent a path
- 🔄 `transfer_status` — poll to SUCCEEDED

🔒 Guardrails, before the call

Eagle writes only under `/eagle/alcf_training/`; 500 files; 1 GiB.

A tool is a decision about what *not* to expose — and `stage_to_alcf` returns a task ID, not a result.

Hands-on



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Hands-on 1: explore read-only

Find the wrong assumptions before one costs node-hours



You

“Status of Polaris and Crux right now, and how many jobs are queued under `alcf_training`?”



Agent

Called `get_system_status` on both, then `list_jobs` filtered by `accountingId`. Both up; 4 jobs queued.

What to watch for

- Which tool did it pick — the cheapest?
- Did it invent a resource ID, or list first?
- Does it admit when it cannot tell?

Try to break it

- Ask about a system that is not there
- Ask for someone else's jobs
- Ask what the tools cannot answer

Hands-on 2: submit a real job

A plain-language request becomes a monitored PBS job

Scientist

"Run this script on 2 Polaris nodes under alcf_training in the reservation, and tell me when it's done."

Agent

Resolved the account and reservation queue, built the job spec, submitted via `submit_job`, then polled until it left the queue.

 the tool call it made

```
submit_job(system="polaris", commands="bash run.sh", nodes=2,  
           stdout_path="/home/<you>/run.out", queue="<reservation>")
```

✔ **Agent:** Job 1234567 finished in 4 min — stdout at ~/log_example.out.

When the job fails

The interesting half — and the one that judges the agent

The demo path is the easy path. Judge an agent on the other one.

✗ It will fail

Wrong filesystem flag, bad account, walltime too short, module not loaded — the ordinary ways a first submission dies.

🔍 Make it diagnose

Ask for the *evidence*: which line of stderr, which field of the job record. Not “it failed, retrying.”

👤 Keep the veto

Let it propose the fix; you approve the resubmit. An agent that retries silently burns your allocation.

A confident wrong answer is the failure mode that matters.

Judge the agent on whether it shows you the line it read — not on whether it sounds sure.

Hands-on 3: generate locally, stage, then run

The first exercise where the file does not already exist at ALCF

>_ make a sweep worth moving

```
mkdir -p ~/ses-staging/sweep
for t in 300 400 500 600 700 800; do
  printf 'temperature = %s\n' "$t" \
    > ~/ses-staging/sweep/run_$(t).in
done
```



You

"Stage ~/ses-staging/sweep to my ALCF home, confirm all six arrived, then run a job that cats each one."

👁 What you are grading

- 🔄 **The verify step.** Correct: stage, poll transfer_status to SUCCEEDED, *then* submit.
- ✗ A task ID is not a delivery. An agent that submits straight away runs against an empty directory — and the failure surfaces minutes later in PBS stderr, looking like a job bug.

🔧 Stop GCP mid-run (-stop) and see whether it notices — or reports success anyway.

Hands-on 4: capture it as a skill

So a hard-won correction survives past the end of the session

 ~/.claude/skills/

```
skills/  
`-- polaris-job/  
    |-- SKILL.md
```

```
---  
name: polaris-job  
description: Submit and monitor a  
             PBS job on Polaris via the IRI API.  
---
```

```
Always list resources first.  
Never resubmit without asking.
```

Why a skill beats a good prompt

-  The description is the routing signal — it is how the agent decides to load the skill at all.
-  Hard-won corrections (“use home:eagle”) stop being retyped every session.
-  A skill is a text file. It reviews, diffs and shares like any other artifact.

The smallest useful unit of institutional memory for an agent.

Where this goes



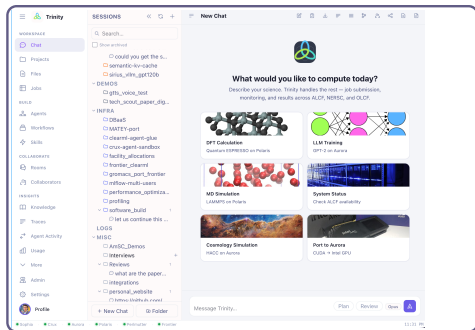
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Where this goes

One agent, many facilities, persistent context



Trinity: this pattern as a multi-user service.

What scales up from today

- More facilities.** One MCP server per site — ALCF, NERSC, OLCF — behind one agent.
- Campaigns, not calls.** 600+ autonomous runs across 13 DOE systems.
- Per-user isolation.** Agents act under the owner's own tokens and budget.
- Shared rooms.** Humans and agents in one conversation.

The same architecture across AmSC

Only the model gateway changes — the IRI half is already shared

At ALCF today

-  Model: **ALCF Inference Service**
-  Compute: **IRI API** + Globus Compute
-  Auth: `alcf-tokens` / Globus OAuth2
-  Reach: Polaris, Crux

Across AmSC

-  Model: **Model Access Gateway (MAG)**
-  Compute: **IRI API** — same REST surface
-  Auth: MAG token + per-facility Globus
-  Reach: Polaris, Perlmutter, Frontier

ALCF is fully committed to AmSC. Today shows what is possible at ALCF now — and the model half swaps cleanly.

Take it home

Everything in this session runs from your own laptop

Repositories

This workshop

argonne-lcf/Service_Enabled_Science

Deeper agent labs (ALCF)

zhenghh04/alcf_agentic_ai

Multi-facility version (AmSC)

zhenghh04/amsc_agentic_ai

Good next steps

-  Add one tool for *your* workload
-  Write one skill for what you retype weekly
-  Run the training lab: MNIST on Polaris
-  Tell us what broke — it is week one

The agent is the new layer. Everything underneath it you already used in Sessions 01–03.

Acknowledgement

This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory and is based on research supported by the U.S. DOE Office of Science-Advanced Scientific Computing Research Program, under Contract No. DE-AC02-06CH11357.

Thanks to the SES organizing team — Tom Uram, Christine Simpson, Benoit Côté, Michael Salim, Ethan Wong.

huihuo.zheng@anl.gov